

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\agent-a95b4528ddf71d428.jsonl`
- SHA-256: `e91a994f6a899d32161b568ba3ddd77b0463ee8aed9d4bc045afdd1cd424179c`
- Source modified: `2026-06-21T19:09:48+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `subagents`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T19:08:04.077Z)

Adversarially review PR P2d (PACKAGING_PLAN.md) in the git worktree C:/Users/avidu/Projects/bhi-wt-p2d. It adds DB upgrade-safety for an auto-updating downloadable app: a downgrade guard + backup-before-migrate + a backup_database() export.

Read in that worktree:

- backend/infrastructure/database.py: the new `SCHEMA\_VERSION` constant + `SchemaTooNewError`, the guard + `\_backup\_before\_migrate` call inside `\_init\_db` (search those names), `\_backup\_before\_migrate`, and `backup\_database`. Also read `\_connect`/`\_get\_connection` (WAL) and the migration ladder for context.
- tests/test_db_upgrade_safety.py

Already verified: full suite 1331 passed/7 skipped, ruff+mypy clean, coverage 85.18% (database.py 92%); the existing v1?reopen upgrade test (test_async_jobs.py) + the ==7 schema-version tests still pass.

Adversarially check, with file:line evidence:

1. DOWNGRADE GUARD: `if version > SCHEMA\_VERSION: raise SchemaTooNewError`. It runs in `\_init\_db` (called from `\_\_init\_\_`), so `Database(newer\_db)` raises. Is SCHEMA_VERSION (7) correct vs the ladder's max migration? Could the guard ever fire on a LEGITIMATE current/older DB (false positive)? Does raising in `\_\_init\_\_` leave a half-open connection / leaked file handle (the `with self.\_connect()` context)?
2. BACKUP-BEFORE-MIGRATE: only when `0 < version < SCHEMA\_VERSION`. Is `PRAGMA wal\_checkpoint(FULL)` then `shutil.copy2(db\_path)` correct for a WAL DB ? does it capture a COMPLETE snapshot (or could -wal/-shm content be missed)? Is copying while the conn is open safe? Is the best-effort WARN-not-block behaviour right (a backup failure must not brick the app)? Could the .bak-vN file collide/accumulate problematically? Is `from\_version` interpolated safely into the filename?
3. backup_database(dest): checkpoints on a fresh _connect then copies. Is the checkpoint effective (it runs on a connection that's then closed ? does the checkpoint persist to the main file)? makedirs parent ? safe? Returns dest.
4. INTERACTION: does the backup/guard interact with the P0e single-instance lock (Database is constructed AFTER the lock in main.py) or break the fresh-DB path (version 0 ? no backup, no guard)? Any behaviour change for the normal fresh/current DB case?
5. EDGE: a corrupt/zero-byte DB file (version reads 0?) ? behaves how? A DB on a read-only FS (WAL fails, copy fails) ? guard/backup degrade gracefully?

Return ONLY: findings as "[SEVERITY blocker|major|minor|nit] <file:line> ? <issue> ? <fix>", then a one-line verdict. Be precise; don't invent issues. If clean, say so.

2. assistant (2026-06-21T19:08:07.593Z)

I'll review PR P2d in the worktree. Let me start by reading the relevant files.

3. user (2026-06-21T19:08:09.648Z)

```
f01dc85 feat(db): v7 ? record the UI locale per job (PMF analytics) + count_jobs_by_locale (#306)
fa7ee3f feat(pkg): P2a ? single ffmpeg/ffprobe resolver (env-overridable; one fail-loud place) (default-OFF) (#305)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
ca34787 feat(pkg): P2b ? friendly `indexer --app` launcher (boot localhost server + open the bundled UI) (#303)
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-OFF) (#302)
---STATUS---
On branch feat/packaging-p2d-db-safety
Your branch is up to date with 'origin/master'.
```

Changes not staged for commit:

```
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified:   backend/infrastructure/database.py
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
tests/test_db_upgrade_safety.py
```

no changes added to commit (use "git add" and/or "git commit -a")

4. user (2026-06-21T19:08:09.867Z)

```
1      """DB upgrade-safety contract for an auto-updating downloadable app (PACKAGING_PLAN.md
P2d)."""
2
3      import os
4      import sqlite3
5
6      import pytest
7
8      from backend.infrastructure.database import SCHEMA_VERSION, Database, SchemaTooNewError
9
10
11     def _user_version(path: str) -> int:
12         conn = sqlite3.connect(path)
13         try:
14             return int(conn.execute("PRAGMA user_version").fetchone()[0])
15         finally:
16             conn.close()
17
18
19     def _set_user_version(path: str, v: int) -> None:
20         conn = sqlite3.connect(path)
21         try:
22             conn.execute(f"PRAGMA user_version = {int(v)}")
23             conn.commit()
24         finally:
25             conn.close()
26
27
28     def test_schema_version_matches_ladder(tmp_path):
29         """Ties the SCHEMA_VERSION constant to the migration ladder: a freshly-built DB
lands exactly at
30         SCHEMA_VERSION. Adding a _migrate_to_vN without bumping the constant fails here."""
31         p = str(tmp_path / "t.db")
32         Database(p)
33         assert _user_version(p) == SCHEMA_VERSION
34
35
36     def test_fresh_db_makes_no_backup(tmp_path):
37         Database(str(tmp_path / "t.db"))
```

```

38         assert not any(".bak" in f.name for f in tmp_path.iterdir()) # version 0 ? nothing
to back up
39
40
41     def test_reopen_current_db_no_backup_no_error(tmp_path):
42         p = str(tmp_path / "t.db")
43         Database(p) # ? SCHEMA_VERSION
44         Database(p) # reopen at current version ? no migration runs
45         assert not any(".bak" in f.name for f in tmp_path.iterdir())
46
47
48     def test_downgrade_guard_refuses_newer_db(tmp_path):
49         p = str(tmp_path / "t.db")
50         Database(p)
51         _set_user_version(p, SCHEMA_VERSION + 5) # a DB from a hypothetical newer app
52         with pytest.raises(SchemaTooNewError):
53             Database(p)
54
55
56     def test_backup_before_migrate_on_upgrade(tmp_path):
57         p = str(tmp_path / "t.db")
58         db = Database(p)
59         db.add_video("vid1", "/a.mp4", "processed", []) # data that must survive
60         _set_user_version(p, 3) # simulate an older on-disk
schema
61
62         Database(p) # reopen with current code ? upgrades in place, backing up first
63         backup = tmp_path / "t.db.bak-v3"
64         assert backup.exists(), "an upgrade must snapshot the DB to <db>.bak-v<old> first"
65         assert _user_version(p) == SCHEMA_VERSION # upgraded
66         assert _user_version(str(backup)) == 3 # the backup is the PRE-upgrade
snapshot
67         assert Database(p).get_video("vid1") is not None # data preserved across the
upgrade
68
69
70     def test_backup_database_export(tmp_path):
71         p = str(tmp_path / "t.db")
72         db = Database(p)
73         db.add_video("vid1", "/a.mp4", "processed", [])
74         dest = str(tmp_path / "export" / "my_data.db")
75         assert db.backup_database(dest) == dest
76         assert os.path.exists(dest) # parent dir created
77         assert _user_version(dest) == SCHEMA_VERSION
78         assert Database(dest).get_video("vid1") is not None # the export is a usable DB
79

```

5. user (2026-06-21T19:08:10.045Z)

```

1     import os
2     import shutil
3     import sqlite3
4     import json
5     import logging
6     from contextlib import contextmanager
7     from typing import List, Dict, Any, Optional, Tuple
8
9     from backend import billing # pure cost math (no backend imports ? no cycle)
10
11     logger = logging.getLogger(__name__)
12
13     # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14     # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,

```

```

15     # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16     # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17     # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18     SCHEMA_VERSION = 7
19
20
21     class SchemaTooNewError(RuntimeError):
22         """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24     class Database:
25         def __init__(self, db_path: str):
26             self.db_path = db_path
27             self._init_db()
28
29         @contextmanager
30         def _connect(self):
31             """Transaction scope that also CLOSES the connection.
32
33             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
34             never closes, so every call leaked its connection to GC (lingering file
35             handles on the WAL sidecars under Windows). Internal methods use this.
36             """
37             conn = self._get_connection()
38             try:
39                 with conn:
40                     yield conn
41             finally:
42                 conn.close()
43
44         def _get_connection(self):
45             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
46             # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
47             conn = sqlite3.connect(self.db_path, timeout=30.0)
48             conn.row_factory = sqlite3.Row
49             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
50             conn.execute("PRAGMA foreign_keys = ON")
51             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
52             conn.execute("PRAGMA busy_timeout = 30000")
53             try:
54                 conn.execute("PRAGMA journal_mode = WAL")
55             except sqlite3.OperationalError:
56                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
57             return conn
58
59         def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
60             """Run ONE bool-returning single-statement write inside a connection scope.
61
62             Captures the connect/commit/except-logger.error/return-False boilerplate shared
63             by
64
65             the bool-returning single-statement writers. ``error_msg`` is the caller's exact
66             per-method message (already formatted with its own ids) ? logged verbatim on a
67             swallowed write fault so each writer keeps its specific log text. Returns True
68             on
69             success, False (after logging ``error_msg``) on any exception."""
70             try:
71                 with self._connect() as conn:
72                     conn.cursor().execute(sql, params)
73                     conn.commit()
74                 return True
75             except Exception as e:
76                 logger.error(f"{error_msg}: {e}")

```

```

74         return False
75
76     @staticmethod
77     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
78         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
79
80         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
81         transaction, so an interrupted v3/v4 block can leave the column added while
82         ``user_version`` stays un-bumped ? and the next open would re-run the same
83         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
84         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
85         Swallow ONLY that specific error so the ladder stays crash-safe, matching the re-
86         runnable ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
87         """
88         try:
89             cursor.execute(ddl)
90         except sqlite3.OperationalError as exc:
91             if "duplicate column name" not in str(exc).lower():
92                 raise
93
94     def _init_db(self):
95         """Initializes tables for videos, play_segments, and events."""
96         with self._connect() as conn:
97             cursor = conn.cursor()
98
99             self._create_base_tables(cursor)
100
101             # Versioned migrations live here. PRAGMA user_version is the schema
102             # contract ? bump it for every change and add an ordered `if version < N`
103             # block below. Tests assert the expected version, so accidental drift fails
104             CI.
105             version = cursor.execute("PRAGMA user_version").fetchone()[0]
106
107             # P2d upgrade-safety (no-op for a fresh DB or one already at
108             # SCHEMA_VERSION):
109             # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
110             # schema this
111             # binary can't reason about; opening it could corrupt data, so refuse
112             # loudly.
113             if version > SCHEMA_VERSION:
114                 raise SchemaTooNewError(
115                     f"This database (schema v{version}) was created by a newer version
116                     of the app "
117                     f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
118                     point it at "
119                     f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}")
120             # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
121             # in place; a
122             # failed migration could corrupt it, so snapshot it first (best-effort:
123             # WARN, never
124             # block the upgrade). version 0 = a brand-new DB ? nothing to back up.
125             if 0 < version < SCHEMA_VERSION:
126                 self._backup_before_migrate(conn, version)
127
128             if version < 1:
129                 self._migrate_to_v1(cursor)
130
131             if version < 2:
132                 self._migrate_to_v2(cursor)
133
134             if version < 3:
135                 self._migrate_to_v3(cursor)

```

```

129         if version < 4:
130             self._migrate_to_v4(cursor)
131
132         if version < 5:
133             self._migrate_to_v5(cursor)
134
135         if version < 6:
136             self._migrate_to_v6(cursor)
137
138         if version < 7:
139             self._migrate_to_v7(cursor)
140         # future: if version < 8: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 8
141
142         conn.commit()
143
144     def _backup_before_migrate(self, conn: sqlite3.Connection, from_version: int) ->
None:
145         """Snapshot the DB to ``db>.bak-v<from_version>`` before an upgrade migration
(P2d).
146
147         Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
148         failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
refusing to
149         start because a backup couldn't be written would be worse than proceeding
without one)."""
150         backup = f"{self.db_path}.bak-v{from_version}"
151         try:
152             conn.execute("PRAGMA wal_checkpoint(FULL)")
153             shutil.copy2(self.db_path, backup)
154             logger.info("[DB] backed up v%s database before upgrading ? %s",
from_version, backup)
155         except (OSError, sqlite3.Error) as exc:
156             logger.warning("[DB] could not back up the database before migrating from
v%s (%s); "
157                             "proceeding with the in-place upgrade.", from_version, exc)
158
159     def backup_database(self, dest: str) -> str:
160         """Write a consistent copy of the DB to ``dest`` (an 'export my data'
affordance, P2d).
161
162         Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
163         parent = os.path.dirname(os.path.abspath(dest))
164         if parent:
165             os.makedirs(parent, exist_ok=True)
166         with self._connect() as conn:
167             conn.execute("PRAGMA wal_checkpoint(FULL)")
168             shutil.copy2(self.db_path, dest)
169             return dest
170
171     def _create_base_tables(self, cursor):
172         """Create the videos / play_segments / events base tables (idempotent)."""
173         # Videos Table
174         cursor.execute("""
175             CREATE TABLE IF NOT EXISTS videos (
176                 id TEXT PRIMARY KEY,
177                 filepath TEXT NOT NULL,
178                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
179                 status TEXT NOT NULL,
180                 validation_results TEXT
181             )
182         """)
183
184         # Play Segments Table (Extensible metadata JSON)
185         cursor.execute("""

```

```

185         CREATE TABLE IF NOT EXISTS play_segments (
186             id INTEGER PRIMARY KEY AUTOINCREMENT,
187             video_id TEXT NOT NULL,
188             start_time REAL NOT NULL,
189             end_time REAL NOT NULL,
190             duration REAL NOT NULL,
191             server TEXT,
192             receiver TEXT,
193             metadata TEXT,
194             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
195         )
196     """
197
198     # Events Table
199     cursor.execute("""
200         CREATE TABLE IF NOT EXISTS events (
201             id INTEGER PRIMARY KEY AUTOINCREMENT,
202             segment_id INTEGER NOT NULL,
203             timestamp REAL NOT NULL,
204             type TEXT NOT NULL,
205             player TEXT,
206             details TEXT,
207             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
208         )
209     """)
210
211     def _migrate_to_v1(self, cursor):
212         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
213         # Common fields are columns; detector-specific metrics go in `stats` JSON,
214         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
215         cursor.execute("""
216             CREATE TABLE IF NOT EXISTS candidate_segments (
217                 id INTEGER PRIMARY KEY AUTOINCREMENT,
218                 video_id TEXT NOT NULL,
219                 detector TEXT NOT NULL,
220                 chunk_index INTEGER,
221                 start_time REAL NOT NULL,
222                 end_time REAL NOT NULL,
223                 duration REAL NOT NULL,
224                 confidence REAL,
225                 stats TEXT,
226                 detection_params TEXT,
227                 confirmed_segment_id INTEGER,
228                 human_label TEXT,
229                 notes TEXT,
230                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
231                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
232                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
233             DELETE SET NULL
234             )
235         """)
236         cursor.execute("""
237             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
238             ON candidate_segments(video_id, detector)
239         """)
240         cursor.execute("PRAGMA user_version = 1")
241
242     def _migrate_to_v2(self, cursor):
243         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
244         # /api/process request. `status` is the EXECUTION state of the job
245         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
246         # that failed validation) lives inside `result` JSON as result["status"].
247         # `result` is the full sync-era response dict (incl. report paths), so the
248         # observability report is the job's artifact, per the plan.
249         cursor.execute("""

```

```

249         CREATE TABLE IF NOT EXISTS jobs (
250             id TEXT PRIMARY KEY,
251             video_path TEXT NOT NULL,
252             video_id TEXT,
253             segmenter TEXT,
254             player_pool TEXT,
255             max_frames INTEGER,
256             status TEXT NOT NULL DEFAULT 'queued',
257             progress TEXT,
258             error TEXT,
259             result TEXT,
260             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
261             started_at TIMESTAMP,
262             finished_at TIMESTAMP
263         )
264         """)
265     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
266     cursor.execute("PRAGMA user_version = 2")
267
268     def _migrate_to_v3(self, cursor):
269         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
270         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
271         # due
272         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
273         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
274         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
275         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
276         # them.
277         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
278         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
279         TIMESTAMP")
280         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
281         next_poll_at)")
282         cursor.execute("PRAGMA user_version = 3")
283
284     def _migrate_to_v4(self, cursor):
285         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
286         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
287         # path can scope rows to an owner. **NULL = local install = byte-identical to
288         # today** ? every existing row stays NULL and every existing query (which never
289         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
290         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
291         # them.
292         # Additive ONLY: no served-behavior change rides on this slice.
293         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
294         TEXT")
295         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
296         user_id TEXT")
297         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
298         # of index churn while making the eventual per-user lookups cheap.
299         cursor.execute(
300             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
301             "WHERE user_id IS NOT NULL")
302         cursor.execute(
303             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
304             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
305         cursor.execute("PRAGMA user_version = 4")
306
307     def _migrate_to_v5(self, cursor):
308         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
309         # version): the resolved per-user `fusion_config` overlay + an INLINE
310         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
311         # evidence
312         # and provenance that earned it. `is_active` pins the one row the serving bridge
313         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This

```



```

slice
306         # only persists/loads it ? NOTHING reads it on the served path yet.
307         #
308         #     user_id            ? owner of the model (NULL allowed = the local install)
309         #     version            ? monotonic per-user model version
310         #     baseline_version   ? the shared baseline this was fit against (upgrade
switch)
311         #     feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
312         #     fusion_config      ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
313         #     classifier_json     ? JSON LogisticRegression.to_dict() (None = config-only
model)
314         #     promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
315         #     n_videos_at_fit     ? held-out-user corpus size when fit (min_n trust floor
input)
316         #     is_active          ? the one resolved row for this user (partial-unique
below)
317         cursor.execute("""
318             CREATE TABLE IF NOT EXISTS user_models (
319                 id INTEGER PRIMARY KEY AUTOINCREMENT,
320                 user_id TEXT,
321                 version INTEGER NOT NULL DEFAULT 1,
322                 baseline_version TEXT,
323                 feature_schema_hash TEXT,
324                 fusion_config TEXT,
325                 classifier_json TEXT,
326                 promotion_evidence TEXT,
327                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
328                 is_active INTEGER NOT NULL DEFAULT 0,
329                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
330             )
331         """)
332         # One model version per user (NULLs compare distinct in SQLite, so the local
333         # install can still carry multiple versioned rows; the active-row guard below
334         # is what enforces a single served model).
335         cursor.execute("""
336             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
337                 ON user_models(user_id, version)
338         """)
339         # At most ONE active model per user: a partial-unique index over the active
rows.
340         # (SQLite treats NULL user_id rows as distinct, so the local install's single
341         # active row is still guarded ? there is one local user.)
342         cursor.execute("""
343             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
344                 ON user_models(user_id) WHERE is_active = 1
345         """)
346         cursor.execute("PRAGMA user_version = 5")
347
348     def _migrate_to_v6(self, cursor):
349         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
350         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
351         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
352         # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
353         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNS are
354         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
355         for col, typ in (

```

```

356         ("owner_id", "TEXT"),          # who pays (the non-owner user / tenant)
357         ("compute_backend", "TEXT"),    # local_gpu | cloud_run | ...
358         ("gpu_type", "TEXT"),           # L4 | T4 | ... (selects the per-second
GPU rate)
359         ("gpu_active_seconds", "REAL"),
360         ("wall_seconds", "REAL"),
361         ("bytes_out", "INTEGER"),        # egress
362         ("gemini_cost_usd", "REAL"),     # provider-reported, already USD
363         ("pricebook_version", "TEXT"),
364         ("cost_usd", "REAL"),            # computed total (USD = source of truth)
365         ("cost_breakdown_json", "TEXT"), # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
366     ):
367         self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
368         # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
369         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
370                        "WHERE owner_id IS NOT NULL")
371         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
372         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
373         cursor.execute("""
374             CREATE TABLE IF NOT EXISTS pricebook (
375                 version TEXT NOT NULL,
376                 resource TEXT NOT NULL,
377                 gpu_type TEXT NOT NULL DEFAULT '',
378                 unit_price_usd REAL NOT NULL DEFAULT 0.0,
379                 currency TEXT NOT NULL DEFAULT 'USD',
380                 effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
381             )
382         """)
383         cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
384                        "ON pricebook(version, resource, gpu_type)")
385         # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
386         # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
387         # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
388         for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
389                                    ("wall_seconds", ""), ("egress_gb", "")):
390             cursor.execute(
391                 "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
392                 "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
gpu_type))
393             cursor.execute("PRAGMA user_version = 6")
394
395     def _migrate_to_v7(self, cursor):
396         # v7: record the UI LOCALE a job was submitted in (the language the SPA was
rendered in,
397         # e.g. "kn", "hi-IN"). NULLABLE ? NULL = unrecorded = byte-identical to today
(CLI / older
398         # clients / pre-v7 rows). Enables PMF analytics ("which language markets use the
tool",
399         # count_jobs_by_locale) and pairs with the localized reel watermark (the SPA
sends the same
400         # locale to /api/compile). Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
401         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN locale TEXT")
402         cursor.execute("PRAGMA user_version = 7")
403
404     def add_video(self, video_id: str, filepath: str, status: str, validation_results:

```

```

List[Dict[str, Any]]) -> bool:
405         """Register or update a video row WITHOUT touching its child segments.
406
407         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
408         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
409         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
410         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
411         row in place; clearing segments is now an explicit, deliberate operation
412         (clear_video_data / prune_segments).
413         """
414         return self._execute_write(
415             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
416             "ON CONFLICT(id) DO UPDATE SET "
417             "filepath=excluded.filepath, status=excluded.status, "
418             "validation_results=excluded.validation_results",
419             (video_id, filepath, status, json.dumps(validation_results)),
420             "Failed to add video",
421         )
422
423     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
424         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
425
426         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
427         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
428         """
429         with self._connect() as conn:
430             cur = conn.cursor()
431             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
432                             (video_id,)).fetchone()[0]
433             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
434                             (video_id,)).fetchone()[0]
435             return int(p), int(c)
436
437     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
438                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
439                       cand_after: Optional[int] = None) -> None:
440         """Delete play/candidate segments by id range (events cascade from
play_segments).
441
442         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
443         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
444         and keep the OLD ones when the run failed/produced nothing (`*_after`).
445         """
446         with self._connect() as conn:
447             cur = conn.cursor()
448             if play_upto is not None:
449                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
450             if play_after is not None:
451                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
452             if cand_upto is not None:
453                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
454             if cand_after is not None:
455                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >

```

```

?", (video_id, cand_after))
456         conn.commit()
457
458     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
459         try:
460             with self._connect() as conn:
461                 cursor = conn.cursor()
462                 cursor.execute(
463                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
464                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
465                 )
466                 conn.commit()
467                 return cursor.lastrowid
468         except Exception as e:
469             logger.error(f"Failed to add play segment: {e}")
470             return None
471
472     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
473         return self._execute_write(
474             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
475             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
476             "Failed to add event",
477         )
478
479     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
480                             chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
481                             stats: Optional[Dict[str, Any]] = None,
482                             detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
483         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
484         detector-specific dicts stored as JSON. Returns the new candidate id, or
485         None."""
486         try:
487             with self._connect() as conn:
488                 cursor = conn.cursor()
489                 cursor.execute(
490                     """INSERT INTO candidate_segments
491                     (video_id, detector, chunk_index, start_time, end_time, duration,
492                     confidence, stats, detection_params)
493                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
494                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
495                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
496                 )
497                 conn.commit()
498                 return cursor.lastrowid
499         except Exception as e:
500             logger.error(f"Failed to add candidate segment: {e}")
501             return None
502
503     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
504         """Record that a candidate detection was confirmed as a given play_segment."""
505         return self._execute_write(
506             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
507             (segment_id, candidate_id),
508             f"Failed to link candidate {candidate_id} to segment {segment_id}",

```

```

508         )
509
510     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
511                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
512         """Fetch candidate detections for the annotation / fine-tuning workflow.
513         `stats` and `detection_params` are deserialized from JSON."""
514         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
515         params: List[Any] = [video_id]
516
517         if detector:
518             query += " AND detector = ?"
519             params.append(detector)
520         if only_unlabeled:
521             query += " AND human_label IS NULL"
522
523         query += " ORDER BY start_time"
524
525         candidates = []
526         with self._connect() as conn:
527             rows = conn.cursor().execute(query, params).fetchall()
528             for row in rows:
529                 d = dict(row)
530                 d["stats"] = json.loads(d["stats"] or "{}")
531                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
532                 candidates.append(d)
533         return candidates
534
535     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
536         with self._connect() as conn:
537             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
538 (video_id,)).fetchone()
539             if row:
540                 d = dict(row)
541                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
542                 return d
543             return None
544
545     def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
546         """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
547 insertion
548 counter, since id is a TEXT PK) rather than the 1-second-granularity
549 processed_at, which
550 would tie within a batch. Mirrors get_video's row shape (validation_results
551 parsed). Powers
552 GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
553         sql = "SELECT * FROM videos ORDER BY rowid DESC"
554         params: tuple = ()
555         if limit is not None:
556             sql += " LIMIT ?"
557             params = (int(limit),)
558         with self._connect() as conn:
559             rows = conn.cursor().execute(sql, params).fetchall()
560             out: List[Dict[str, Any]] = []
561             for row in rows:
562                 d = dict(row)
563                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
564                 out.append(d)
565             return out
566
567     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
568 List[Dict[str, Any]]:
569         """
570         Dynamically queries play segments based on filters:
571         - duration_min: float
572         - server: string

```

```

568         - receiver: string
569         - event_type: string (e.g. smash, foul)
570         """
571         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
572         params = [video_id]
573
574         if filters.get("duration_min"):
575             query += " AND r.duration >= ?"
576             params.append(filters["duration_min"])
577
578         if filters.get("server"):
579             query += " AND r.server = ?"
580             params.append(filters["server"])
581
582         if filters.get("receiver"):
583             query += " AND r.receiver = ?"
584             params.append(filters["receiver"])
585
586         if filters.get("event_type"):
587             # Check if there is an event of a specific type inside this segment
588             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
589             params.append(filters["event_type"])
590
591         segments_list = []
592         with self._connect() as conn:
593             cursor = conn.cursor()
594             rows = cursor.execute(query, params).fetchall()
595
596             for row in rows:
597                 r_dict = dict(row)
598                 r_id = r_dict["id"]
599                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
600
601                 # Fetch associated events
602                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
603                 r_dict["events"] = []
604                 for er in event_rows:
605                     e_dict = dict(er)
606                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
607                     r_dict["events"].append(e_dict)
608
609                 segments_list.append(r_dict)
610
611         return segments_list
612
613     def clear_video_data(self, video_id: str):
614         with self._connect() as conn:
615             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
616             conn.commit()
617
618     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
619     # Writers follow the repo convention (swallow + logger.error, return bool); the
620     # worker logs loudly on a failed transition so a job can't silently wedge.
621
622     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
623                   player_pool: Optional[List[str]] = None,
624                   max_frames: Optional[int] = None,
625                   policy: Optional[Dict[str, Any]] = None,
626                   owner_id: Optional[str] = None,
627                   locale: Optional[str] = None) -> bool:
628         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
629         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.

```

```

``owner_id``
630         (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
631         ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded."""
632         return self._execute_write(
633             "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
634                 "owner_id, locale) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?)",
635             (job_id, video_path, segmenter,
636              json.dumps(player_pool) if player_pool is not None else None,
637              max_frames, json.dumps(policy) if policy is not None else None, owner_id,
638              locale),
639             f"Failed to create job {job_id}",
640         )
641     def count_jobs_by_locale(self) -> Dict[str, int]:
642         """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
643         CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
644         with self._connect() as conn:
645             rows = conn.cursor().execute(
646                 "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
647                 "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
648             ).fetchall()
649         return {str(r[0]): int(r[1]) for r in rows}
650
651     def mark_job_running(self, job_id: str) -> bool:
652         return self._execute_write(
653             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
654             "progress='starting' WHERE id = ?", (job_id,),
655             f"Failed to mark job {job_id} running",
656         )
657
658     def set_job_progress(self, job_id: str, progress: str) -> bool:
659         return self._execute_write(
660             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
661             f"Failed to set progress for job {job_id}",
662         )
663
664     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
665         return self._execute_write(
666             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
667             f"Failed to set video_id for job {job_id}",
668         )
669
670     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
671         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
672         validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
673         return self._execute_write(
674             "UPDATE jobs SET status='done', progress='finished', result=?, "
675             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
676             (json.dumps(result), job_id),
677             f"Failed to mark job {job_id} done",
678         )
679
680     def mark_job_failed(self, job_id: str, error: str) -> bool:
681         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
682         return self._execute_write(
683             "UPDATE jobs SET status='failed', error=?, "
684             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
685             f"Failed to mark job {job_id} failed",
686         )

```

```

687
688     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
689         """Fetch one job; `result`/`player_pool` JSON deserialized."""
690         with self._connect() as conn:
691             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
692             if not row:
693                 return None
694             d = dict(row)
695             d["result"] = json.loads(d["result"]) if d["result"] else None
696             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
697             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
698             return d
699
700     # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
-----
701     def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) ->
Dict[str, float]:
702         """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
703         GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=None``.
704         An unknown version ? ``{}`` (so every cost computes to 0.0)."""
705         gt = gpu_type or ""
706         rates: Dict[str, float] = {}
707         with self._connect() as conn:
708             rows = conn.cursor().execute(
709                 "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version
= ?",
710                 (version,)).fetchall()
711             for r in rows:
712                 res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
713                 if res == billing.GPU_SECONDS:
714                     if row_gt == gt:         # the GPU rate for THIS gpu_type
715                         rates[res] = price
716                 else:
717                     rates[res] = price         # wall/egress: gpu_type-independent
718             return rates
719
720     def record_job_cost(self, job_id: str, *, usage: Dict[str, float],
pricebook_version: str,
721                        gpu_type: Optional[str] = None, compute_backend: Optional[str] =
None,
722                        owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
723         """Compute the job's cost from ``usage`` x the pricebook + persist every cost
column.
724         Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
725         ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
726         rates = self.get_pricebook_rates(pricebook_version, gpu_type)
727         # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
728         # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
729         # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
730         gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
731         unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
732         if unpriced_gpu:
733             logger.warning(
734                 "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-

```



```

seconds billed "
735             "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
736             gpu_type, pricebook_version, gpu_amt)
737 cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
738 if unpriced_gpu:
739     breakdown["unpriced_gpu_seconds"] = gpu_amt
740 if margin:
741     marked = billing.apply_margin(cost_usd, margin)
742     breakdown["margin"] = round(marked - cost_usd, 6) # keep ?(breakdown) ==
cost_usd
743     cost_usd = marked
744 bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
745 ok = self._execute_write(
746     "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
747     "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
748     "cost_breakdown_json=? WHERE id = ?",
749     (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
750      usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
751      pricebook_version, cost_usd, json.dumps(breakdown), job_id),
752     f"Failed to record cost for job {job_id}")
753 if not ok:
754     return None
755 return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
756        "pricebook_version": pricebook_version}
757
758 def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
759     """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
``None`` until
760     recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
761     with self._connect() as conn:
762         row = conn.cursor().execute(
763             "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
764             "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd,
"
765             "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
766     if not row:
767         return None
768     d = dict(row)
769     raw = d.pop("cost_breakdown_json", None)
770     d["cost_breakdown"] = json.loads(raw) if raw else {}
771     return d
772
773 def get_video_cost(self, video_id: str) -> Dict[str, Any]:
774     """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
A video is
775     1:1 with an infer job, but a re-ingest can produce several ? sum them."""
776     with self._connect() as conn:
777         rows = conn.cursor().execute(
778             "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
779             "WHERE video_id = ?", (video_id,)).fetchall()
780     jobs: List[Dict[str, Any]] = []
781     total = 0.0
782     for r in rows:
783         cost = r["cost_usd"]
784         if cost is not None:
785             total += float(cost)
786         jobs.append({
787             "job_id": r["id"], "cost_usd": cost, "pricebook_version":
r["pricebook_version"],
788             "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
r["cost_breakdown_json"] else {},

```

```

789         })
790         return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
791
792     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
793     -----
794     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
795         """Attach/replace a job's JSON ExecutionPolicy."""
796         return self._execute_write(
797             "UPDATE jobs SET policy=? WHERE id = ?",
798             (json.dumps(policy) if policy is not None else None, job_id),
799             f"Failed to set policy for job {job_id}",
800         )
801
802     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
803     None) -> bool:
804         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
805         releasing the
806         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
807         ``next_poll_at`` is
808         computed in SQLite's own UTC format so it compares directly to
809         CURRENT_TIMESTAMP."""
810         return self._execute_write(
811             "UPDATE jobs SET status='waiting', "
812             "next_poll_at=datetime('now', ?), "
813             "progress=COALESCE(?, progress) WHERE id = ?",
814             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
815             f"Failed to set job {job_id} waiting",
816         )
817
818     def seconds_until_next_due(self) -> Optional[float]:
819         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
820         can
821         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
822         is
823         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
824         (0.0)."""
825         try:
826             with self._connect() as conn:
827                 row = conn.cursor().execute(
828                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
829                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
830                     secs "
831                     "FROM jobs WHERE status='waiting'").fetchone()
832             if row is None or row["secs"] is None:
833                 return None
834             return max(0.0, float(row["secs"]))
835         except Exception as e:
836             logger.error(f"Failed to compute next-due delay: {e}")
837             return None
838
839     def claim_due_job(self) -> Optional[Dict[str, Any]]:
840         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
841         ``next_poll_at`` is due ?
842         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
843         never
844         double-claim (the DB row-claim is the only coordination needed; no master).
845         Earliest-due
846         first. Returns the claimed job dict, or None if nothing is runnable."""
847         try:
848             with self._connect() as conn:
849                 cur = conn.cursor()
850                 row = cur.execute(
851                     "SELECT id, status FROM jobs WHERE status='queued' "
852                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
853                     CURRENT_TIMESTAMP)) "

```

```

841         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
842         if not row:
843             return None
844         cur.execute(
845             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
846             "WHERE id=? AND status=?", (row["id"], row["status"]))
847         conn.commit()
848         if cur.rowcount != 1:
849             return None                    # lost the race to a concurrent
worker
850         return self.get_job(row["id"])
851     except Exception as e:
852         logger.error(f"Failed to claim a due job: {e}")
853         return None
854
855     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
856         """Crash recovery: any job still queued/running from a previous process is dead
857         (the executor is in-process). Mark failed so clients see a terminal state, not a
858         hang. Called once at startup. Returns the number of jobs swept."""
859         try:
860             with self._connect() as conn:
861                 cur = conn.cursor()
862                 cur.execute(
863                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
864                     "WHERE status IN ('queued', 'running')", (reason,))
865                 conn.commit()
866                 return cur.rowcount
867         except Exception as e:
868             logger.error(f"Failed to sweep stale jobs: {e}")
869             return 0
870
871     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
872     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
873     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
874     # serving.py) resolves them, and wiring that bridge into the production decision
seam
875     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
876     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
877     # JSON-serialised dicts; the rest are scalar columns.
878
879     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
880                           baseline_version: Optional[str] = None,
881                           feature_schema_hash: Optional[str] = None,
882                           fusion_config: Optional[Dict[str, Any]] = None,
883                           classifier_json: Optional[Dict[str, Any]] = None,
884                           promotion_evidence: Optional[Dict[str, Any]] = None,
885                           n_videos_at_fit: int = 0,
886                           is_active: bool = False) -> Optional[int]:
887         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
888
889         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
890         the
891         rowid). When `is_active` is True this row becomes the user's single active
892         model ?
893         any previously-active row for the SAME user is first cleared, so the
894         `idx_user_models_one_active` partial-unique invariant always holds (one active
895         model
896         per user). Returns the row id, or None on failure.
897         """
898         try:
899             with self._connect() as conn:
900                 cur = conn.cursor()
901                 if is_active:

```

```

899             # Demote the current active row first (NULL user_id = local: IS NULL
match).
900         if user_id is None:
901             cur.execute(
902                 "UPDATE user_models SET is_active = 0 "
903                 "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
904                 (version,))
905         else:
906             cur.execute(
907                 "UPDATE user_models SET is_active = 0 "
908                 "WHERE user_id = ? AND is_active = 1 AND version != ?",
909                 (user_id, version))
910         cur.execute(
911             """INSERT INTO user_models
912             (user_id, version, baseline_version, feature_schema_hash,
913             fusion_config, classifier_json, promotion_evidence,
914             n_videos_at_fit, is_active)
915             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
916             ON CONFLICT(user_id, version) DO UPDATE SET
917                 baseline_version=excluded.baseline_version,
918                 feature_schema_hash=excluded.feature_schema_hash,
919                 fusion_config=excluded.fusion_config,
920                 classifier_json=excluded.classifier_json,
921                 promotion_evidence=excluded.promotion_evidence,
922                 n_videos_at_fit=excluded.n_videos_at_fit,
923                 is_active=excluded.is_active""",
924             (user_id, version, baseline_version, feature_schema_hash,
925              json.dumps(fusion_config) if fusion_config is not None else None,
926              json.dumps(classifier_json) if classifier_json is not None else
None,
927              json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
928              n_videos_at_fit, 1 if is_active else 0))
929         conn.commit()
930         rid = cur.execute(
931             "SELECT id FROM user_models WHERE "
932             + ("user_id IS NULL" if user_id is None else "user_id = ?")
933             + " AND version = ?",
934             ((version,) if user_id is None else (user_id, version))).fetchone()
935         return int(rid[0]) if rid else None
936     except Exception as e:
937         logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
938     return None
939
940     def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
941         """Return the single active per-user model row for ``user_id`` (NULL = local
install),
942         with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
943         or None if the user has no active model. Does not touch the served pipeline."""
944         with self._connect() as conn:
945             if user_id is None:
946                 row = conn.cursor().execute(
947                     "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
948             else:
949                 row = conn.cursor().execute(
950                     "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
951                     (user_id,)).fetchone()
952             if not row:
953                 return None
954             d = dict(row)
955             for col in ("fusion_config", "classifier_json", "promotion_evidence"):
956                 d[col] = json.loads(d[col]) if d[col] else None

```

```
957         return d
958
```

6. assistant (2026-06-21T19:08:12.973Z)

Now let me check the migration ladder max, the P0e single-instance lock interaction, and how Database is constructed in main.py.

7. user (2026-06-21T19:08:15.040Z)

```
121:         self._migrate_to_v1(cursor)
124:         self._migrate_to_v2(cursor)
127:         self._migrate_to_v3(cursor)
130:         self._migrate_to_v4(cursor)
133:         self._migrate_to_v5(cursor)
136:         self._migrate_to_v6(cursor)
139:         self._migrate_to_v7(cursor)
211:     def _migrate_to_v1(self, cursor):
241:     def _migrate_to_v2(self, cursor):
268:     def _migrate_to_v3(self, cursor):
280:     def _migrate_to_v4(self, cursor):
300:     def _migrate_to_v5(self, cursor):
348:     def _migrate_to_v6(self, cursor):
395:     def _migrate_to_v7(self, cursor):
```

8. user (2026-06-21T19:08:15.979Z)

```
140:         # future: if version < 8: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 8
239:         cursor.execute("PRAGMA user_version = 1")
266:         cursor.execute("PRAGMA user_version = 2")
278:         cursor.execute("PRAGMA user_version = 3")
298:         cursor.execute("PRAGMA user_version = 4")
346:         cursor.execute("PRAGMA user_version = 5")
393:         cursor.execute("PRAGMA user_version = 6")
402:         cursor.execute("PRAGMA user_version = 7")
```

9. user (2026-06-21T19:08:17.398Z)

```
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:47:from backend.utils.single_instance import
acquire_lock # P0e: one instance per database
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:68:     does zero filesystem I/O ? a
hosted/multi-tenant deploy sets this to lock CORS to its origin.""
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:78:# lock it to its origin; the middleware is
fixed at startup.
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:152:     ""For --app: keep `port` if free,
else fall back to a free one so a busy 8000 never blocks the app.""
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:228:# P0e: the single-instance lock handle.
Kept module-global for the process lifetime ? if it were
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:229:# GC'd/closed the OS would release the
lock. None until main() acquires it (server/CLI init).
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:230:_instance_lock: Optional[Any] = None
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:954:     # in-flight jobs to 'failed'. The OS
lock (held for the process lifetime; auto-released on exit
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:956:     global _instance_lock
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:958:         _instance_lock =
acquire_lock(db_path + ".lock")
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:960:         # The lock path isn't writable
(read-only dir / perms). Don't crash and don't falsely claim a
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:961:         # second instance ? warn and
proceed WITHOUT the guard (a lock-infra failure must not block a
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:962:         # legitimate single instance).
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:963:         logger.warning("[JOBS] Could not
create the single-instance lock at %s.lock (%s); proceeding "
```

```

C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:965:         _instance_lock = None
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:967:         if _instance_lock is None:
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:973:         db_instance = Database(db_path)
C:\Users\avidu\Projects\bhi-wt-p2d\backend\main.py:1101:        # imported as `backend.main`,
`__name__` != "__main__", so this block doesn't re-fire.)

```

10. assistant (2026-06-21T19:08:20.773Z)

The ladder max is v7, matching `SCHEMA\_VERSION = 7`. Let me look at the main.py construction order and the single-instance lock to verify the interaction claims.

11. user (2026-06-21T19:08:21.390Z)

```

948         os.makedirs(global_config.output.output_dir, exist_ok=True)
949
950         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
951
952         # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
953         # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
954         # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
955         # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
956         global _instance_lock
957         try:
958             _instance_lock = acquire_lock(db_path + ".lock")
959         except OSError as exc:
960             # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
961             # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
962             # legitimate single instance).
963             logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
964                             "WITHOUT the second-instance guard.", db_path, exc)
965             _instance_lock = None
966         else:
967             if _instance_lock is None:
968                 print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
969                     f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
970                     f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
971                 sys.exit(1)
972
973         db_instance = Database(db_path)
974
975         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
976         # the executor is in-process. Mark them failed so pollers see a terminal state.
977         swept = db_instance.fail_stale_jobs()
978         if swept:
979             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
980
981         # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
982         if args.video_path and not args.server and not args.app:
983             print(f"[CLI] Processing video file: {args.video_path}")
984             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
985             # The local-existence fast-fail only applies to a local input (stat the RESOLVED

```

```
key, not the
986         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
987         storage_cfg = getattr(global_config, "storage", None)
```

12. assistant (2026-06-21T19:08:27.573Z)

The construction order in ``main.py`` confirms: ``acquire_lock`` (line 958) runs before ``Database(db_path)`` (line 973). Now let me verify a subtle WAL detail ? whether ``_backup_before_migrate`` is called inside ``_init_db``'s ``with self._connect()`` block, meaning the checkpoint runs on the same connection that already wrote base tables. Let me also examine whether the checkpoint inside ``_init_db`` can actually flush given that the surrounding transaction is open.

The key concern for #2: ``_backup_before_migrate(conn, version)`` (line 118) runs *inside* the ``with self._connect()`` as `conn:`` block (line 96), and critically it runs after ``_create_base_tables(cursor)`` (line 99) which issued ``CREATE TABLE IF NOT EXISTS`` statements. Let me verify the transaction state carefully.

13. user (2026-06-21T19:08:28.304Z)

```
94         def _init_db(self):
95             """Initializes tables for videos, play_segments, and events."""
96             with self._connect() as conn:
97                 cursor = conn.cursor()
98
99                 self._create_base_tables(cursor)
100
101                 # Versioned migrations live here. PRAGMA user_version is the schema
102                 # contract ? bump it for every change and add an ordered `if version < N`
103                 # block below. Tests assert the expected version, so accidental drift fails
104                 CI.
105                 version = cursor.execute("PRAGMA user_version").fetchone()[0]
106
107                 # P2d upgrade-safety (no-op for a fresh DB or one already at
108                 SCHEMA_VERSION):
109                 # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
110                 schema this
111                 #     binary can't reason about; opening it could corrupt data, so refuse
112                 loudly.
113                 if version > SCHEMA_VERSION:
114                     raise SchemaTooNewError(
115                         f"This database (schema v{version}) was created by a newer version
116                         of the app "
117                         f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
118                         point it at "
119                         f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}")
120                 # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
121                 #     in place; a
122                 #     failed migration could corrupt it, so snapshot it first (best-effort:
123                 WARN, never
124                 #     block the upgrade). version 0 = a brand-new DB ? nothing to back up.
125                 if 0 < version < SCHEMA_VERSION:
126                     self._backup_before_migrate(conn, version)
127
128                 if version < 1:
129                     self._migrate_to_v1(cursor)
130
131                 if version < 2:
132                     self._migrate_to_v2(cursor)
133
134                 if version < 3:
135                     self._migrate_to_v3(cursor)
136
137                 if version < 4:
```

```

130         self._migrate_to_v4(cursor)
131
132     if version < 5:
133         self._migrate_to_v5(cursor)
134
135     if version < 6:
136         self._migrate_to_v6(cursor)
137
138     if version < 7:
139         self._migrate_to_v7(cursor)
140     # future: if version < 8: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 8
141
142     conn.commit()
143
144     def _backup_before_migrate(self, conn: sqlite3.Connection, from_version: int) ->
None:
145         """Snapshot the DB to ``<db>.bak-v<from_version>`` before an upgrade migration
(P2d).
146
147         Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
148         failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
refusing to
149         start because a backup couldn't be written would be worse than proceeding
without one)."""
150         backup = f"{self.db_path}.bak-v{from_version}"
151         try:
152             conn.execute("PRAGMA wal_checkpoint(FULL)")
153             shutil.copy2(self.db_path, backup)
154             logger.info("[DB] backed up v%s database before upgrading ? %s",
from_version, backup)
155         except (OSError, sqlite3.Error) as exc:
156             logger.warning("[DB] could not back up the database before migrating from
v%s (%s); "
157                             "proceeding with the in-place upgrade.", from_version, exc)
158
159     def backup_database(self, dest: str) -> str:
160         """Write a consistent copy of the DB to ``dest`` (an 'export my data'
affordance, P2d).
161         Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
162         parent = os.path.dirname(os.path.abspath(dest))
163         if parent:
164             os.makedirs(parent, exist_ok=True)
165         with self._connect() as conn:
166             conn.execute("PRAGMA wal_checkpoint(FULL)")
167             shutil.copy2(self.db_path, dest)
168         return dest

```

14. assistant (2026-06-21T19:08:33.501Z)

Now I have a complete picture. Let me analyze the critical WAL-checkpoint-while-transaction-open issue carefully. The key subtlety:

In `_init_db`, `_create_base_tables` runs `CREATE TABLE IF NOT EXISTS` (line 99). With Python's `sqlite3` module in default (legacy) `isolation_level` mode, the first DML/DDI implicitly opens a transaction. `CREATE TABLE` is DDL. Whether the `_backup_before_migrate` checkpoint at line 152 actually flushes depends on whether a write transaction is open at that point.

Let me empirically verify this rather than reason about it, since it's the load-bearing correctness claim. Let me write a quick test against the actual code.

15. user (2026-06-21T19:08:45.448Z)

```
version read = 3
```



```
wal_checkpoint(FULL) result (busy, log_frames, checkpointed) = (0, 0, 0)
backup user_version = 3
backup rows = [('vid1', '/a.mp4')]
backup has -wal sidecar? False
```

16. assistant (2026-06-21T19:08:49.583Z)

The checkpoint result `(busy=0, log\_frames=0, checkpointed=0)` confirms the WAL was already empty/fully flushed at this point (the prior connection's `.commit()` + close had already checkpointed), so the single-file copy is complete. Now the critical adversarial case: what if there ARE uncheckpointed frames in the WAL when the backup runs, AND a transaction is open? Let me test that the `wal\_checkpoint(FULL)` blocks on an open write transaction.

17. user (2026-06-21T19:08:59.057Z)

```
after commit, -wal exists? True size 24752
main-file-only rows (reopen fresh, but conn still open):
version read = 3
wal_checkpoint(FULL) with another conn open = (0, 6, 6) (busy flag=1 means it could NOT fully
flush)
backup user_version = 3
backup rows = [('vid1', '/a.mp4')]
```

18. assistant (2026-06-21T19:09:06.506Z)

Even with another connection open and a 24KB un-truncated WAL, `wal\_checkpoint(FULL)` returned `(busy=0, log\_frames=6, checkpointed=6)` ? all 6 frames flushed to the main file before the copy, so the single-file `shutil.copy2` captured a complete snapshot. The single-instance lock (P0e) guarantees no truly concurrent writer in production anyway. This validates the WAL-completeness claim.

One more: the `backup\_database` export checkpoints on a connection that is then **closed** before the copy. Let me verify the checkpoint persists to the main file after that connection closes (it must, since FULL flushes synchronously). The first probe already showed checkpointed frames land in the main file before the copy in `\_backup\_before\_migrate`; for `backup\_database` the checkpoint completes inside `conn.execute` before `with` closes, so the main file is updated. The test `test\_backup\_database\_export` confirms the copy is a usable DB.

Now let me confirm the `from\_version` interpolation safety and `.bak` collision behavior, and check there's no `errors` import or unused concerns. Let me verify the existing v1->reopen test reference and the ==7 schema tests exist as claimed.

19. user (2026-06-21T19:09:07.827Z)

```
tests/test_async_jobs.py:10:- migration: fresh DB lands at user_version 2 with a jobs table; v1
DBs upgrade in place
tests/test_async_jobs.py:83:         assert conn.execute("PRAGMA user_version").fetchone()[0] ==
7
tests/test_async_jobs.py:101:         conn.execute("PRAGMA user_version = 1")
tests/test_async_jobs.py:107:         assert c.execute("PRAGMA user_version").fetchone()[0] == 7
tests/test_database.py:86:         version = conn.execute("PRAGMA user_version").fetchone()[0]
tests/test_database.py:89:         assert version == 7
tests/test_database.py:108:         version = conn.execute("PRAGMA user_version").fetchone()[0]
tests/test_database.py:114:         assert version == 7          # v7 = job UI locale (PMF analytics); v6
= cost ledger (M8)
tests/test_database.py:229:         conn.execute("PRAGMA user_version = 3")
tests/test_database.py:237:         assert conn.execute("PRAGMA user_version").fetchone()[0] == 7
tests/test_database.py:255:         on its own, so a crash after the ADD COLUMNS but before the
`PRAGMA user_version`
tests/test_database.py:263:         # but the process died before the version bump, so user_version
is stuck at 2.
tests/test_database.py:266:         conn.execute("PRAGMA user_version = 2")
tests/test_database.py:275:         assert conn.execute("PRAGMA user_version").fetchone()[0] == 7
tests/test_database.py:351:def test_upsert_is_idempotent_on_user_version(tmp_path):
```

```
tests/test_database.py:361:    assert m["n_videos_at_fit"] == 7
```

20. assistant (2026-06-21T19:09:13.988Z)

All referenced tests confirmed. Let me verify one edge case the prompt asked about: the half-open connection on a raised `SchemaTooNewError` in `\_\_init\_\_`. The guard at line 109 raises *inside* the `with self.\_connect() as conn:` block (line 96). The `\_connect` contextmanager's `finally: conn.close()` (line 42) will run during exception unwinding, so the connection is closed ? no leak. Let me confirm there's no scenario where the file handle leaks, and double-check the `.bak` accumulation behavior by re-reading the `\_connect` finally semantics ? already confirmed at lines 38-42.

Let me also verify the zero-byte/corrupt DB edge: `PRAGMA user\_version` on a zero-byte file. SQLite treats an empty file as a valid empty DB (version 0).

21. user (2026-06-21T19:09:19.037Z)

zero-byte size: 0

zero-byte user_version = 0 -> treated as fresh (version 0): no guard, no backup

corrupt -> raises sqlite3.DatabaseError: file is not a database (escapes _init_db, NOT caught by guard/backup)

22. assistant (2026-06-21T19:09:23.890Z)

Edge behavior confirmed: a zero-byte file reads as version 0 (treated as fresh, no guard/no backup ? correct). A corrupt non-SQLite file raises `sqlite3.DatabaseError` from the very first `CREATE TABLE`/`PRAGMA` in `\_init\_db`, which propagates out of `\_\_init\_\_` ? this is *pre-existing* behavior unchanged by P2d (the guard/backup never get a chance to run, and that's fine: there's nothing to back up and nothing to guard).

Let me clean up the probe files and do a final confirmation that nothing in the diff is unused (the `\_backup\_before\_migrate` `conn` param is used; `shutil`/`os` already imported at top).

23. user (2026-06-21T19:09:25.028Z)

```
backend/infrastructure/database.py | 53 ++++++
1 file changed, 53 insertions(+)
```

24. assistant (2026-06-21T19:09:48.191Z)

I have everything needed for the review.

Adversarial review: PR P2d (DB upgrade-safety)

****1. DOWNGRADE GUARD**** ? `SCHEMA\_VERSION = 7` (database.py:18) matches the ladder's max (`\_migrate\_to\_v7` / `PRAGMA user\_version = 7` at database.py:395-402; no `\_migrate\_to\_v8`). The pin test `test\_schema\_version\_matches\_ladder` (test_db_upgrade_safety.py:28) plus the existing `==7` tests enforce drift. ****No false-positive risk**** on a current/older DB: the guard is `version > SCHEMA\_VERSION` (database.py:109), so a v7 DB (`7 > 7` false) and any older DB pass straight through. ****No leaked handle on raise****: the guard raises *inside* `with self.\_connect() as conn:` (database.py:96), and `\_connect`'s `finally: conn.close()` (database.py:42) runs during exception unwind ? verified the connection is closed before `SchemaTooNewError` escapes `\_\_init\_\_`. Clean.

****2. BACKUP-BEFORE-MIGRATE**** ? gating `0 < version < SCHEMA\_VERSION` (database.py:117) is correct (fresh=0 skips, current=7 skips). ****WAL snapshot completeness verified empirically****: `PRAGMA wal\_checkpoint(FULL)` (database.py:152) synchronously flushes all WAL frames into the main file *before* `shutil.copy2` of the single main file ? I reproduced this with a 24KB untruncated WAL and even a second open connection, and the copied-main-file-only backup contained every row + the correct `user\_version`, with no orphaned `-wal` sidecar. The single-instance lock (P0e) also rules out a real concurrent writer in production. Copying while `conn` is open is safe (FULL completed first). Best-effort WARN-not-block (database.py:155-157, catches `OSError`/`sqlite3.Error`) is the right call for a downloadable app. `from\_version` is an `int` read from `PRAGMA user\_version` (database.py:104) ? not user-controlled, no injection/path

concern in the f-string filename (database.py:150). ``.bak-vN`` is keyed by ``from_version`` so each upgrade-from-version writes exactly one file and re-running overwrites idempotently ? bounded, no unbounded accumulation.

****3. backup_database(dest)**** ? checkpoint runs on the fresh ``_connect`` and the FULL checkpoint completes *synchronously inside* ``conn.execute`` (database.py:166) before the ``with`` closes, so the flush is durably in the main file by the time ``shutil.copy2`` (database.py:167) runs after close. ``os.makedirs(parent, exist_ok=True)`` guarded by ``if parent`` (database.py:163-164) handles a bare filename safely. ``test_backup_database_export`` confirms the export is a usable DB at ``SCHEMA_VERSION``. Returns ``dest``. Correct.

****4. INTERACTION**** ? ``acquire_lock`` (main.py:958) runs before ``Database(db_path)`` (main.py:973), so the guard/backup execute under the single-instance lock as intended ? no concurrent-writer hazard during checkpoint/copy. Fresh-DB path (version 0): guard skipped (``0 > 7`` false), backup skipped (``0 < 0`` false) ? ``test_fresh_db_makes_no_backup`` + ``test_reopen_current_db_no_backup_no_error`` confirm zero behavior change for the normal fresh/current case.

****5. EDGE**** ? Zero-byte file: reads ``user_version = 0`` ? treated as fresh, no guard, no backup (verified). Corrupt non-SQLite file: raises ``sqlite3.DatabaseError`` from the first statement in ``_init_db`` ? but this is *pre-existing* behavior unchanged by P2d (base-table creation already ran before this PR), and there's nothing to guard/back-up anyway. Read-only FS: WAL pragma already degrades via the ``except sqlite3.OperationalError: pass`` at database.py:55-56; backup failure degrades via the WARN path (database.py:155). Both degrade gracefully.

Findings

No blocker/major/minor issues found.

- ``[nit] backend/infrastructure/database.py:155`` ? ``_backup_before_migrate`` catches ``(OSError, sqlite3.Error)``; an unexpected non-OSError/non-sqlite exception during checkpoint/copy (e.g. a ``shutil`` edge or a ``MemoryError``) would propagate and brick ``__init__``, defeating the "never block the upgrade" intent. Extremely unlikely in practice given copy2/checkpoint failure modes are ``OSError/sqlite3.Error``. Fix (optional): broaden to ``except Exception`` for true best-effort, matching the repo's swallow-and-log convention used by the job writers. ? Judgment call; the narrow catch is also defensible.

****Verdict:**** Clean ? the downgrade guard, WAL-complete backup (empirically verified), and export are correct and well-tested; ship it. (One optional nit on the backup exception breadth.)